CS3244: Final Project Report (Group 6)

Machine Learning Approaches for Electricity Fraud Detection

Alfreda Goh | Alyssa Khoo | Bryan Kee | Ha Phuong | Minh Thang | Wee En

GitHub Repository: https://github.com/KryanBee/cs3244-fraud-detection.git

1. Introduction & Problem Framing

Background Information:

Electricity fraud is a significant source of non-technical loss for utility providers, arising from activities such as meter tampering, illegal connections, and billing manipulation. These losses impose substantial financial costs, reduce grid reliability, and indirectly increase tariffs for legitimate consumers. Detecting such fraud is therefore both an economic and operational priority. Traditional detection systems are typically rule-based, relying on heuristics such as abrupt changes in consumption or deviations from historical patterns. However, these approaches are limited in their ability to generalize. Legitimate fluctuations in usage can trigger false positives, while more sophisticated fraudulent behavior that mimics normal consumption may go undetected. Machine learning offers a data-driven alternative by learning patterns directly from historical consumption data and capturing more complex relationships. However, its effectiveness depends critically on how well the underlying data challenges (particularly class imbalance) are addressed.

The Challenge of Severe Class Imbalance:

The primary challenge in this task is the extreme imbalance between fraudulent and non-fraudulent cases. In the STEG dataset, only 7,566 out of 135,493 clients are fraudulent (approximately 5.6%), corresponding to a class ratio of roughly 17:1. This imbalance creates two fundamental issues. From a **learning perspective**, most models are biased toward the majority class, as minimizing overall error favors correct classification of non-fraud cases. As a result, models often fail to learn meaningful patterns for the minority class, leading to consistently low recall for fraud detection. From an **evaluation perspective**, standard metrics such as accuracy become misleading. A no-skill classifier that predicts all instances as non-fraud would achieve 94.4% accuracy while failing to detect any fraudulent cases. This demonstrates that accuracy does not reflect task performance in imbalanced settings. To address this, evaluation must prioritize metrics that focus on the minority class. In this project, Precision-Recall AUC (PR-AUC) is used as the primary model selection criterion, as it captures the trade-off between precision and recall without requiring a fixed threshold. Threshold-dependent metrics such as F1-score and recall are used for final evaluation to assess practical performance.

Project Objective:

This project aims to understand and diagnose model performance under severe class imbalance rather than simply optimize predictive scores. We adopt an iterative approach, starting from baseline models and progressively introducing methods to address imbalance. Specifically, we examine three classes of techniques: **data-level resampling**, **cost-sensitive learning**, and more expressive models such as **boosting-based ensembles**. All experiments are conducted within a unified evaluation pipeline to ensure fair comparison and reproducibility. By analyzing how each method affects model behavior, we identify key limitations and attribute performance changes to specific design choices. This approach provides a more principled understanding of fraud detection in imbalanced settings and highlights directions for further improvement.

2. Dataset & Feature Engineering

Dataset Structure:

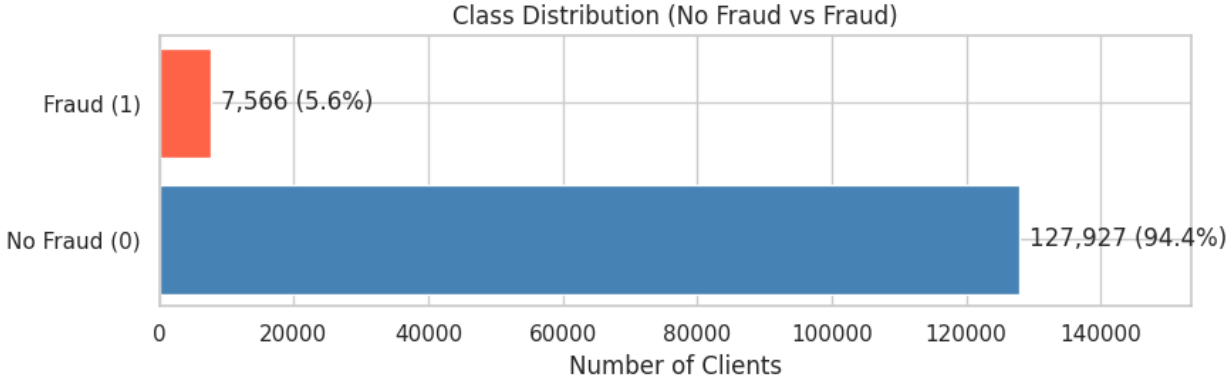
The dataset used in this project is obtained from the Tunisian Company of Electricity and Gas (STEG). It consists of two complementary tables operating at different levels of granularity. The **client table** contains 135,493 entries, where each row corresponds to a unique customer with associated demographic and account information, along with a binary fraud label.

Column	Description
client_id	Unique identifier for each client (string key).
district	Categorical code representing the client's district.
client_catg	Categorical variable indicating the client's category or segment.
region	Categorical code representing the broader geographic region.
creation_date	Date when the client account was registered.
target	Binary fraud label indicating whether the client is fraudulent (1) or legitimate (0).

The **invoice table** contains 4,476,749 records, where each row represents a single billing instance linked to a client via a one-to-many relationship. It includes information on meter identity, operational status, billing periods, and electricity or gas consumption. The 16-column schema can be broadly grouped into meter identity, meter state and readings, and tiered consumption quantities.

Column	Description
client_id	Foreign key linking the invoice to a client record.
invoice_date	Date on which the invoice was issued.
counter_number	Identifier for the physical meter device.
counter_type	Type of energy measured (e.g., electricity or gas).
tarif_type	Tariff plan applied for the billing period.
counter_statue	Categorical indicator of the meter's operational status.
counter_code	Code representing the meter's configuration or model.
counter_coefficient	Scaling factor used to convert raw readings into actual consumption.
reading_remarque	Categorical remark recorded during meter reading.
old_index	Meter reading at the start of the billing period.
new_index	Meter reading at the end of the billing period.
months_number	Duration of the billing period in months.
consommation_level_1	Consumption in the lowest (base) usage tier.
consommation_level_2	Consumption in the second (moderate usage) tier.
consommation_level_3	Consumption in the third (high usage) tier.
consommation_level_4	Consumption in the highest usage tier.

Additionally, the dataset is highly imbalanced, with only 5.6% of clients labelled as fraudulent resulting in a high majority-to-minority ratio of approximately 16.9:1.



Aggregation Methodology:

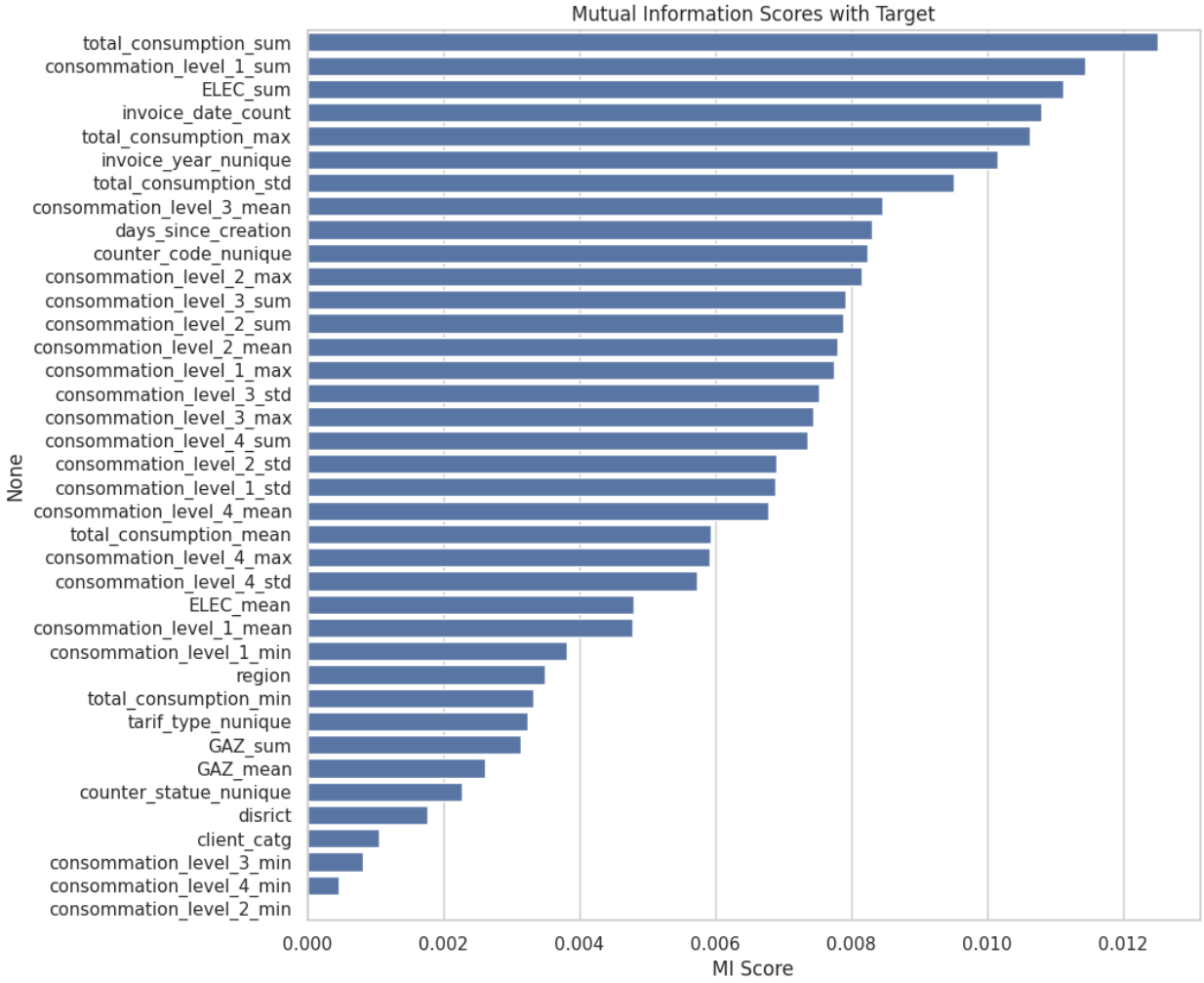
Since all models operate at the client level, the invoice-level data must be aggregated to produce a fixed set of features per client. The aggregation pipeline consists of the following stages:

1. **Account Level Features:** The client's creation_date is converted into two numerical features relative to a reference date (31 December 2019): days_since_creation and months_active.
2. **Invoice-Level Temporal Features:** From invoice_date, we derive temporal features including invoice_year, invoice_month, and a binary is_weekend indicator to capture seasonal patterns.
3. **Data cleaning:** Inconsistent alphanumeric values in counter_statue are standardized into a uniform ordinal representation to reduce noise.
4. **Invoice-level engineered features:** Several features are computed at the invoice level
    - total_consumption: sum of all four tiered consumption quantities
    - monthly_consumption: total_consumption / months_number
    - is_zero_consumption: binary indicator for periods with no recorded usage
    - delta_index: new_index - old_index
    - delta_time: number of days between consecutive invoices
    - consumption_change: first-order difference in total_consumption across invoices
5. **Statistical Aggregation:** Numerical features are aggregated using five summary functions: mean, standard deviation, maximum, minimum, and sum, resulting in a total of 65 features. Additional aggregated features include transactions_count, invoice_year_nunique, and categorical cardinality counts for meter-related attributes.
6. **Energy-Type Breakdown:** Consumption features are aggregated separately by energy type (e.g., ELEC_consumption_sum). Ratios between electricity and gas usage are also derived, which may help identify irregularities such as bypassing a specific meter channel.
7. **Volatility Features:** To capture irregular consumption patterns, the following features are computed for each consumption tier and delta_index.
    - Range: max - min
    - Max-to-mean ratio: max / (mean + eps); flags extreme spikes against a modest baseline
8. **Final merge and invoice frequency:** The aggregated features are merged with the client-level dataset. A composite feature, invoice_frequency = transactions_count / months_active, is introduced to capture billing regularity.

Feature Selection & Analysis:

Given the relatively large feature space (~100 features), several techniques were used to assess feature relevance and redundancy. **Mutual Information** (MI) was first applied as a filter method to measure statistical dependency between each feature and the target variable without assuming linearity. A small number of features with near-zero MI scores (< 0.001) were initially identified as weak predictors and considered for removal. However, subsequent experiments showed that removing these features did not lead to any meaningful improvement in model performance. As a result, all features were retained to preserve as much information as possible.

In addition, **correlation analysis** was conducted to identify redundancy among features. Several highly collinear feature pairs were observed, particularly among engineered consumption-related variables (e.g., *_max vs *_range, total_consumption vs ELEC_consumption, days_since_creation vs months_active). For linear models, these redundant features were removed to reduce instability caused by multicollinearity. However, for tree-based models, no features were dropped, as such models are generally robust to collinearity and can effectively handle redundant inputs. Furthermore, most features were found to have weak correlation with the target variable, suggesting that the relationship between inputs and fraud detection is largely non-linear.



Overall, these analyses indicate that while some features may appear weak in isolation, collectively they contain useful information. Therefore, the final approach prioritizes preserving the full feature set, allowing more expressive models, particularly tree-based and ensemble methods, to learn these non-linear relationships more effectively.

Limitations of Feature Representation:

Despite extensive feature engineering, several limitations remain.

1. **Loss of temporal information:** Aggregating invoice-level data into summary statistics removes sequential structure, making it difficult to capture temporal patterns such as abrupt consumption changes or irregular usage trends.
2. **Limited domain knowledge:** Certain categorical variables (e.g., reading_remarque, counter_code) are treated as generic features without domain-specific interpretation, limiting their ability to provide meaningful predictive signals.
3. **Weak and noisy signal:** Due to severe class imbalance, fraudulent behaviour is both rare and subtle. Aggregation further dilutes this signal by combining fraudulent and legitimate patterns within the same client history.
4. **Aggregation granularity:** Operating at the client level prevents detection of fine-grained, invoice-level anomalies, which may be more indicative of fraudulent activity in real-world scenarios

3. Experimental Pipeline & Evaluation

This study adopts a unified experimental pipeline to ensure reproducibility, fair model comparison, and unbiased evaluation. The pipeline standardizes data handling, model training, and evaluation across all experiments, allowing differences in performance to be attributed solely to model design.

Evaluation Strategy:

Due to severe class imbalance, accuracy is not an appropriate metric, as it is dominated by the majority class. Instead, **Precision-Recall Area Under the Curve** (PR-AUC) is used as the primary evaluation metric, as it focuses on performance on the minority (fraud) class without requiring a fixed classification threshold. The evaluation follows a two-stage framework:

1. **Threshold-independent evaluation:** Models and hyperparameters are selected using cross-validated PR-AUC. This ensures fair comparison without bias from arbitrary threshold choices.
2. **Threshold-dependent evaluation:** A decision threshold is selected separately using out-of-fold (OOF) predictions on the training data. Final performance is then reported on a held-out test set using Precision, Recall, F1-score, and PR-AUC.

Cross Validation Design:

All experiments use Stratified 5-Fold cross-validation, which preserves the class distribution in each fold. This is critical under class imbalance, ensuring that each training and validation split reflects the true fraud rate. To ensure reproducibility, all random processes are controlled using a fixed random seed to guarantee consistent results across repeated runs. Hyperparameter tuning is performed using grid search within the cross-validation framework, ensuring that model selection is based solely on validation performance.

Pipeline Architecture & Data Leakage Prevention:

A key design objective is to eliminate data leakage, which can lead to overly optimistic performance estimates. All preprocessing and optional resampling steps are encapsulated within a unified pipeline. When SMOTE techniques are enabled, they are applied strictly within training folds during cross-validation. Validation and test data remain untouched, preserving the original class distribution. This ensures that:

- Synthetic samples are never generated using validation or test data
- Model evaluation reflects realistic deployment conditions
- Comparisons between models (with and without resampling) remain fair

Out-of-Fold Threshold Selection:

A critical component of the pipeline is the use of out-of-fold (OOF) predictions for threshold selection. Instead of selecting a threshold on the test set (which would introduce bias), predicted probabilities are generated for each training instance using models that did not see that instance during training. This produces an unbiased estimate of model confidence. An optimal decision threshold is then determined based on these OOF predictions. Because this process is entirely confined to the training data, it avoids overfitting and ensures that the test set remains a true hold-out benchmark.

Final Model Training & Evaluation:

After hyperparameter tuning and threshold selection, the best model is retrained on the full training dataset using the selected configuration. Final evaluation is performed on a held-out test set, which is never used during training, tuning, or threshold selection. This provides an unbiased estimate of real-world performance. In addition to test metrics, cross-validation statistics (mean and standard deviation of PR-AUC) are reported to assess model stability. All experiment outputs (trained models, configurations, and evaluation metrics) are systematically saved, ensuring full reproducibility of results.

4. Baseline Models & Analysis of Limitations

Baseline Performance Overview:

To establish a performance reference point, baseline models were evaluated on the original feature set without resampling. All models perform above a no-skill baseline, defined by ROC-AUC = 0.5 and PR-AUC equal to the positive class prevalence (0.056). However, they show a consistent limitation in discriminating against the positive class. This indicates that class imbalance and limited feature representation are key issues affecting performance across all models. Notably, the similarity between cross-validation and test PR-AUC suggests that the models are not overfitting, but are instead underfitting the data.

Model (Baseline)	ROC AUC (test)	PR AUC (test)	F1 Score (test)	Precision (test)	Recall (test)	PR AUC (cv mean)	PR AUC (cv std)
Decision Tree	0.826	0.283	0.358	0.333	0.386	0.271	0.00981
K Nearest Neighbors	0.809	0.223	0.149	0.081	0.946	0.207	0.00582
Logistic Regression	0.818	0.202	0.308	0.229	0.473	0.209	0.0117
Linear Support Vector Machine	0.811	0.216	0.323	0.236	0.510	0.224	0.0136

K-Nearest Neighbors (KNN):

The KNN model exhibits the weakest overall performance. Its predictions are heavily skewed towards the positive class, resulting in extremely high recall but very low precision, indicating a large number of false positives. This imbalance leads to a poor F1-score and reflects the model's inability to form meaningful decision boundaries. A key contributing factor is the **curse of dimensionality**, as the dataset contains over 100 features. In such settings, distance metrics become less informative and the notion of "nearest neighbors" degrades, making it difficult for KNN to effectively distinguish between classes.

Linear Models:

Logistic Regression and Linear Support Vector Machine demonstrate similar behavior, with modest improvements in F1-score compared to KNN but no meaningful gains in PR-AUC. Both models rely on linear decision boundaries, which limits their ability to capture complex relationships in the data. This limitation is supported by PCA and LDA projections, which show substantial overlap between classes and **no clear linear separation**. In addition, the Linear SVM was found to be **computationally impractical** for this task. Due to its training complexity and the dataset size of approximately 100,000 samples, model training was prohibitively slow. Despite attempts to optimize runtime with reduced folds and parallel processing, the performance gains were marginal, and the model was therefore not pursued further.

Decision Tree:

The Decision Tree achieves the strongest performance among the baseline models, as it is able to capture nonlinear relationships, feature interactions, and threshold-based patterns without relying on linear assumptions or distance metrics. However, its performance remains limited, indicating that it still fails to fully capture the underlying patterns of the minority class. While decision trees are generally sensitive to variations in the data, the low cross-validation variance observed here suggests relatively stable performance in this setting. Despite this, the model remains insufficient due to its limited capacity to reduce bias and fully capture the complexity of the data. Nevertheless, its relatively stronger performance suggests that tree-based approaches are more suitable for this problem setting, motivating further exploration of more expressive variants such as ensemble methods.

5. Tackling Class Imbalance

Motivation & Approach:

The baseline results show a systematic inability across all models to effectively distinguish the minority positive class. This is most likely driven by severe class imbalance, which biases learning toward the majority class and limits the models' exposure to minority patterns. As a result, models exhibit high bias and fail to learn discriminative features for fraud detection. To address this, we evaluate a range of imbalance handling techniques spanning both **data-level approaches**, which modify the training distribution, and **algorithm-level approaches**, which adjust the learning objective.

Technique	Methodology	Limitations	Decision Rationale
Random Oversampling	Duplicates existing minority samples until classes are balanced	At a 16.9:1 ratio, repeated samples increase the risk of overfitting by encouraging the model to memories minority instances	**Ruled out** due to high overfitting risk under extreme imbalance
Random Undersampling	Randomly removes majority samples to balance class distribution	Discards a large portion of data (~120k samples), leading to potential information loss and underfitting	**Ruled out** as excessive information loss outweighs potential benefits
SMOTE	Generates synthetic minority samples by interpolating between a sample and its k-nearest minority neighbors	May introduce noise by oversampling in low-information regions and near class overlap boundaries	**Tested**, as it expands the minority feature space beyond simple duplication
Borderline SMOTE	Identifies minority samples near decision boundaries and generates synthetic samples only from these points	Limited oversampling if few samples lie near the boundary which may not sufficiently enrich the minority class	**Tested**, as it focuses on informative boundary regions to improve class separation and reduce false positives compared to SMOTE
SMOTE Tomek	Applies SMOTE, then removes Tomek links (nearest neighbor majority-minority pairs) to clean class boundaries	May remove informative borderline majority samples and introduces additional preprocessing complexity	**Tested**, to assess whether boundary cleaning improves separability after oversampling
Class Weighting	Scales the loss for minority samples by $\frac{n_{samples}}{2 \cdot n_{minority}}$, increasing the penalty for misclassifying fraud cases without altering the data distribution	Minority samples remain sparse, limiting the model's exposure to diverse fraud patterns	**Tested**, as it avoids synthetic data and integrates naturally as a tunable hyperparameter

Experimental Setup and & Results:

Imbalance handling techniques were evaluated on two representative models: **Decision Tree**, as the strongest baseline non-linear model, and **Logistic Regression**, as a representative linear model. KNN was not considered as it does not support class weighting due to its non-parametric nature, while SVM was excluded due to computational constraints. This setup enables a fair comparison of imbalance strategies across both linear and non-linear model classes.

Model	Technique	ROC-AUC	PR-AUC	F1-Score	Precision	Recall
Decision Tree	Class Weights	0.840	0.289	0.381	0.315	0.483
	SMOTE	0.825	0.222	0.326	0.253	0.458
	Borderline SMOTE	0.829	0.249	0.336	0.313	0.363
	SMOTE Tomek	0.825	0.222	0.326	0.253	0.458
Logistic Regression	Class Weights	0.812	0.218	0.319	0.256	0.422
	SMOTE	0.818	0.203	0.307	0.225	0.483
	Borderline SMOTE	0.817	0.204	0.312	0.220	0.537
	SMOTE Tomek	0.817	0.203	0.311	0.219	0.535

Analysis of Imbalance Handling Methods:

SMOTE and its variants do not lead to significant improvements over the baseline. In fact, performance generally deteriorates, as both F1-score and PR-AUC are lower than the baseline, indicating a worse trade-off between precision and recall. Compared to class weighting, SMOTE-based methods achieve slightly higher recall but at the cost of substantially lower precision. This suggests that these techniques make the model more sensitive to detecting fraud cases by over-predicting the positive class, resulting in a large number of false positives. Several factors could explain why SMOTE is ineffective in this setting:

1. **Weak feature representation:** Aggregated features dilute temporal and behavioral patterns, so interpolating between samples can produce unrealistic synthetic points that do not reflect meaningful fraud behavior.
2. **High-dimensional feature space**: In a high-dimensional setting, nearest neighbors may not be truly similar, making SMOTE interpolation less reliable and increasing noise.
3. **Sparse and heterogeneous minority class**: Fraud cases are diverse and do not form clear clusters, so interpolation may mix unrelated patterns and further blur class boundaries.

In contrast, class weighting consistently improves performance across both Decision Tree and Logistic Regression. In both cases, it achieves higher PR-AUC and F1-score than the baseline and all SMOTE variants, indicating a better balance between precision and recall. This improvement across both linear and non-linear models suggests that the effect is model-agnostic. By increasing the penalty for misclassifying fraud cases, class weighting encourages the model to better utilize the limited minority signal without altering the data distribution.

Given these results, class weighting is selected as the primary imbalance handling strategy moving forward. Although the performance gains are modest, they are consistent and achieved without introducing synthetic noise, making class weighting a more reliable approach for this dataset. However, despite these improvements, the overall performance remains limited and the models continue to exhibit signs of high bias. Notably, the Decision Tree remains the strongest performing model, reinforcing the importance of non-linear methods, but its capacity is still insufficient to fully capture the underlying patterns in the data.

This motivates the next stage of the project, where more expressive models, particularly boosting-based ensemble methods, are explored. These methods aim to reduce bias by iteratively focusing on difficult examples, and when combined with class weighting, offer a promising direction for improving minority class detection.

6. Boosting-Based Models

Motivation & Approach:

Given the limited gains from imbalance handling, the main issue appears to be high model bias rather than variance. Although class weighting provided small improvements in PR-AUC and F1-score, overall performance remained constrained, suggesting insufficient model capacity to capture complex fraud patterns. As Decision Tree was the strongest baseline, we extend this approach using boosting-based variants, which reduce bias by combining multiple weak learners into a more expressive model. We evaluate three boosting algorithms; each trained with and without class weights.

1. **Adaptive Boosting (AdaBoost):** Increases the importance of misclassified samples through iterative reweighting, making subsequent trees focus more on difficult cases.
2. **Histogram-Based Gradient Boosting:** Improves efficiency by discretizing continuous features into bins and optimizing residual errors by gradient descent. This scales well with large datasets.
3. **eXtreme Gradient Boosting (XGBoost):** Further extends this framework with regularization and advanced optimization techniques, allowing for better generalization and control over overfitting.

Boosting Model	Class Weights	ROC AUC (test)	PR AUC (test)	F1 Score (test)	Precision (test)	Recall (test)	PR AUC (cv mean)	PR AUC (cv std)
Adaptive Boosting Adaptive Boosting	Non-Weighted	0.827	0.209	0.296	0.205	0.536	0.205	0.0118
	Weighted	0.662	0.083	0.159	0.088	0.838	0.0813	0.0017
Histogram-Based Gradient Boosting	Non-Weighted	0.874	0.351	0.407	0.357	0.473	0.344	0.0147
	Weighted	0.872	0.343	0.402	0.334	0.505	0.336	0.0139
eXtreme Gradient Boosting	Non-Weighted	0.876	0.357	0.406	0.349	0.485	0.349	0.0119
	Weighted	0.874	0.356	0.410	0.353	0.490	0.347	0.0107

Adaptive Boosting (AdaBoost):

AdaBoost performs worse than the Decision Tree baseline, and the introduction of class weighting leads to a substantial decline in performance across all metrics. While the weighted variant achieves very high recall, this comes at the cost of extremely low precision, indicating that the model heavily over-predicts the positive class. This behavior can be explained by the nature of AdaBoost itself, which already increases the weight of misclassified samples during training. Adding class weights on top of this results in an overemphasis on minority class instances, causing the model to focus excessively on difficult or noisy fraud cases. As a result, it loses its ability to discriminate effectively, leading to a surge in false positives and overall degraded performance.

Histogram-Based Gradient Boosting:

Histogram-Based Gradient Boosting shows a significant improvement over the baseline Decision Tree, achieving much higher PR-AUC and F1-score. This indicates that boosting successfully increases model capacity and allows for better learning of complex patterns. However, a similar pattern emerges with class weighting, where the weighted model achieves slightly higher recall but at the expense of lower precision, resulting in a marginal decline in PR-AUC and overall performance. This suggests that the boosting process itself is already sufficiently focused on hard-to-classify examples through gradient updates, and additional class weighting may introduce unnecessary bias toward the minority class. As a result, the unweighted model provides a better balance between precision and recall.

eXtreme Gradient Boosting (XGBoost):

XGBoost delivers the strongest overall performance among all models, with the highest PR-AUC and F1-score observed in the experiments. Both weighted and non-weighted variants perform very similarly, indicating that class weighting has minimal impact on this model. This is likely because XGBoost inherently incorporates mechanisms such as gradient-based optimization and regularization, which allow it to effectively handle class imbalance without requiring explicit reweighting. Its ability to focus on difficult examples while maintaining control over overfitting enables it to achieve a well-balanced trade-off between precision and recall. The comparable performance between the two variants suggests that the model is robust to imbalance and can learn meaningful patterns directly from the data.

Overall Findings:

Boosting methods substantially improve performance, with Histogram-Based Gradient Boosting and XGBoost achieving much higher PR-AUC and F1-scores than the baseline Decision Tree. This represents the largest gain in the project and suggests that increasing model capacity is more effective than further tuning imbalance handling techniques. XGBoost performs best, with strong and stable results across both weighted and non-weighted settings. However, a consistent pattern across all boosting models is that **class weighting provides little to no benefit and can even degrade performance**, particularly for AdaBoost. Unlike simpler models where weighting clearly helped, boosting methods already focus on hard-to-classify samples through sequential learning. Adding class weights can therefore overemphasize the minority class, increase false positives and degrade overall performance.

7. Conclusion

Overall, this project highlights the inherent difficulty of electricity fraud detection in highly imbalanced and noisy real-world datasets. While traditional imbalance handling techniques are often emphasized in theory, our findings show that their effectiveness is highly context-dependent. In particular, SMOTE and its variants did not improve performance and in many cases degraded it, likely due to the high-dimensional feature space and weak, heterogeneous minority signal, where synthetic samples introduce more noise than useful information. Class weighting proved to be more reliable, yielding consistent improvements for simpler models by encouraging greater sensitivity to fraud cases without altering the data distribution. However, the most significant performance gains were achieved not through imbalance handling, but through the adoption of more expressive models. Boosting-based methods, especially gradient boosting and XGBoost & Histogram-Based Gradient Boosting, substantially improved PR-AUC, F1-Score and overall classification performance, suggesting that model capacity and the ability to capture complex, non-linear patterns are more critical than data-level adjustments in this setting. Notably, these models were also largely robust to class imbalance without requiring explicit weighting, reinforcing the idea that algorithmic design can subsume some of the benefits of traditional techniques.

Despite these improvements, the final model still reflects the fundamental challenges of the task. With a PR-AUC of 0.356, precision of approximately 35%, and recall around 50%, the model is clearly better than a naive baseline but remains limited in practical effectiveness. In particular, the relatively low PR-AUC indicates that adjusting the decision threshold to meet different operational needs would be non-trivial, as gains in recall are likely to come at a steep cost in precision. These results suggest that the primary bottleneck lies not only in modeling, but also in feature representation and data quality. Future work could explore more advanced approaches, such as deep learning models that better capture temporal dynamics, or richer feature engineering incorporating domain knowledge. Ultimately, this project underscores that in complex, imbalanced problems like fraud detection, improving model expressiveness often yields greater returns than relying solely on standard imbalance correction techniques.

8. Appendix

Statement of Independent Work:

By entering our Student IDs below, we certify that this assignment was completed independently, in accordance with the CS3244 class policy, except for the resources explicitly acknowledged below. All ideas, analysis, and conclusions presented in this report are the original work of the group. We followed the CS3244 class policy in completing this assignment and have no exceptions to declare.

Signed,
A0192401R, A0266282Y, A0281511N, A0282949J, A0283248X, A0338991L

References:

We acknowledge the following resources that supported our work:

- **Kaggle Notebook: *"4th Place in Fraud Detection from Zindi"***
  https://www.kaggle.com/code/imgremlin/4th-place-in-fraud-detection-from-zindi
  Used as a reference for initial feature engineering ideas and baseline aggregation strategies.
- **ChatGPT and Claude Sonnet**:
  Used during the report-writing phase to assist with structuring content and refining explanations. All outputs were critically reviewed, edited, and validated by the team to ensure accuracy and alignment with project objectives.
- **GitHub Copilot**
  Used during development to assist with code documentation, suggest boilerplate implementations, and support debugging. All generated code suggestions were reviewed and tested to ensure correctness and consistency with the overall system design.